PES UNIVERSITY

SOFTWARE ARCHITECTURE DESIGN

Electronics Store Inventory Management System

Document Detail	Information
Document Version	1.0
Document Status	Draft for Review
Academic Year	2026?2027
Team	Team 15
Prepared By	P. Dhanush
SRN	PES2UG24CS333
Date	3 November 2026

Revision History

Version	Date	Author	Description
1.0	03-11-2026	P. Dhanush	Initial architecture design document

1. Introduction

1.1 Purpose

This Software Architecture Design (SAD) document describes the architecture, major components, interfaces, database organization, and security-related design of the Electronics Store Inventory Management System.

1.2 Scope

The system is a web-based application for maintaining electronic component inventory. It supports user login, dashboard summaries, inventory viewing, component creation and modification, deletion, searching, filtering, stock classification, reports, and CSV export.

1.3 Intended Audience

- Project team members and developers
- Course instructors and evaluators
- Testers and maintainers

1.4 Definitions

Term	Meaning
SAD	Software Architecture Design
UI	User Interface
SKU	Stock Keeping Unit
CRUD	Create, Read, Update and Delete
CSV	Comma-Separated Values
FR	Functional Requirement
NFR	Non-Functional Requirement

2. Architecture Overview

2.1 Architectural Style

The application follows a simple three-layer web architecture. The presentation layer consists of HTML templates and CSS. The application layer is implemented in Python using Flask and handles routing, authentication, validation, inventory operations, reporting, and CSV generation. The data layer uses SQLite for persistent storage.

2.2 Technology Stack

Layer / Purpose	Technology	Responsibility
Backend	Python	Application logic
Web framework	Flask	Routes, request handling, sessions and rendering
Frontend	HTML templates	Web pages and forms
Styling	CSS	Presentation and layout
Database	SQLite	Persistent component and user

		data
Authentication	Werkzeug security	Password hashing and verification
Export	Python csv and io	CSV report generation
Testing	Python test suite	Application, frontend and inventory backend tests

2.3 High-Level Architecture

A user interacts with the web interface through a browser. Flask receives the HTTP request and invokes the relevant route. The application layer performs authentication and validation where required, communicates with SQLite through database connection functions, and returns rendered HTML or a CSV response.

Logical flow: User/Browser ? Flask Routes and Application Logic ? SQLite Database. Flask renders HTML templates and serves responses back to the browser.

2.4 Design Constraints

- The current application uses SQLite as its database.
- The backend is implemented in a single main application file, app.py.
- The user interface is based on server-rendered HTML templates.
- The application uses environment variables for configurable database path, secret key, and default low-stock threshold.

3. Component Design

3.1 Component Inventory

Component	Implementation	Description
Authentication	app.py: login, logout, login_required	Authenticates users and restricts protected routes.
Dashboard	dashboard route and dashboard.html	Displays inventory summary counts.
Inventory	inventory route and inventory.html	Displays component records and supports search and filters.
Component Management	add_component.html, edit_component.html and related routes	Creates, updates and deletes inventory records.
Stock Classification	get_status()	Determines Available, Low Stock or Out of Stock status.
Reporting	report route and report.html	Displays available, low-stock and out-of-stock component groups.
CSV Export	export_csv route	Generates a downloadable CSV response.
Database Access	get_db_connection(), initialize_database()	Connects to SQLite and initializes or migrates tables.
Presentation	templates/ and static/css/style.css	Provides HTML pages and styling.

3.2 Component Responsibilities

The Flask application coordinates requests and application operations. Template files define the pages presented to users. The database stores component and user records. The CSS file controls visual presentation. Test files exercise application routes, frontend behavior, and inventory backend logic.

3.3 Component Interaction

1. The user submits a request from a browser page.
2. Flask matches the request to an application route.
3. The route checks authentication where required and validates submitted data.
4. The application reads or changes data using SQLite.
5. The route renders an HTML template or returns a CSV response.

4. Database Architecture

4.1 Database Overview

The application uses a local SQLite database file named database.db by default. The database connection uses sqlite3.Row for named column access and enables SQLite foreign-key enforcement.

4.2 Components Table

Column	Type	Description / Constraint
component_id	INTEGER	Primary key; auto-increment
sku	TEXT	Unique stock keeping unit
component_name	TEXT	Required component name
category	TEXT	Required category
quantity	INTEGER	Required; must be zero or greater
price	REAL	Required; must be zero or greater
min_stock_level	INTEGER	Required; default 10; must be zero or greater

4.3 Users Table

Column	Type	Description / Constraint
user_id	INTEGER	Primary key; auto-increment
username	TEXT	Required and unique
password_hash	TEXT	Required hashed password value

4.4 Data Integrity

- Component SKU values are unique.
- Component quantity cannot be negative.
- Component price cannot be negative.
- Minimum stock level cannot be negative.
- Usernames are unique.
- Database initialization includes migration logic for older component tables.

5. Interface Design

5.1 User Interface Pages

Template	Purpose
login.html	User login form
dashboard.html	Inventory summary dashboard
inventory.html	Inventory listing, search and filters
add_component.html	Form to add a component
edit_component.html	Form to update a component
report.html	Stock availability report

base.html	Shared page layout

5.2 Application Routes

HTTP Route	Purpose
/login	Display and process login
/logout	Log out using POST
/	Display dashboard
/inventory	Display inventory
/add	Add a component
/edit/<component_id>	Edit a component
/delete/<component_id>	Delete a component using POST
/report	Display stock report
/export/csv	Export inventory as CSV

5.3 External Interface

The primary external interface is a web browser communicating with the Flask application over HTTP. The application returns rendered HTML pages and a downloadable CSV response. The database is accessed internally by the Python application.

5.4 Error Handling

The application uses form validation, redirects, and Flask flash messages to communicate form or operation outcomes. Database constraints provide additional validation for selected fields. Detailed error behavior should be verified against the current implementation and test results.

6. Security Architecture

6.1 Authentication

- The login route accepts username and password credentials.
- Werkzeug generate_password_hash and check_password_hash are used for password hashing and verification.
- The session stores the authenticated user identifier and username.
- The login_required decorator protects application routes that require authentication.

6.2 Session Management

The application clears the session before setting authenticated session values during login. The logout route clears the session. Logout is implemented as a POST route.

6.3 Security Configuration and Deployment Notes

- The Flask secret key can be supplied through the SECRET_KEY environment variable.
- The current code contains a development-only fallback secret key. A deployment must set a strong secret key through the environment.
- HTTPS configuration and production deployment controls are outside the verified local application behavior and must be configured before deployment.
- Security testing should verify that protected routes reject unauthenticated requests and that passwords are not stored as plaintext.

7. Sequence Diagrams

7.1 Login Sequence

Participants: User, Browser, Flask Login Route, User Database.

6. The user opens the login page.
7. The browser displays the login form.
8. The user submits the username and password.
9. Flask retrieves the corresponding user record.
10. The password hash is verified using Werkzeug.
11. On successful verification, the application establishes the session and redirects the user to the dashboard.
12. On failure, the login page is shown with an appropriate message.

7.2 Add Component Sequence

Participants: User, Browser, Flask Add Route, Validation Logic, SQLite Database.

13. The user opens the Add Component page.
14. The browser displays the component form.
15. The user submits component details.
16. Flask validates the submitted values.
17. The application inserts the new component record into SQLite.
18. The application redirects the user to an appropriate page and displays feedback.

7.3 Inventory Viewing Sequence

19. The user opens the inventory page.
20. Flask receives the inventory request and checks authentication.
21. The application queries component records from SQLite.
22. Search and filter criteria are applied when supplied.
23. Flask renders inventory.html with the matching records.
24. The browser displays the inventory.

8. Quality Attributes

Attribute	Design Consideration
Usability	Consistent templates, forms, navigation and feedback messages.
Maintainability	Separate HTML templates, CSS styling, and named application functions.
Reliability	SQLite constraints and automated tests support correct operations.
Performance	Database queries and server-rendered pages support the current project scale; measurable targets require testing.
Portability	Python and SQLite are available on common development platforms; environment configuration supports different local settings.

9. Risks and Limitations

Risk / Limitation	Design Consideration
Single application module	As the project grows, application logic may be

	separated into modules or blueprints.
SQLite deployment scale	A larger multi-user deployment may require a server database.
Development secret fallback	Set SECRET_KEY securely in the deployment environment.
No verified HTTPS setup	Configure HTTPS at the deployment layer before production use.
Performance target not measured	Conduct performance testing with representative inventory sizes.

10. Requirement Traceability

Requirement Area	Architecture Component
Login and logout	Authentication component and Flask session
Dashboard summary	Dashboard route, database queries and dashboard.html
View inventory	Inventory route, SQLite and inventory.html
Add component	Add route, validation and components table
Edit component	Edit route, validation and components table
Delete component	Delete route and components table
Search and filters	Inventory route and inventory.html
Stock status	get_status() and report logic
Stock report	Report route and report.html
CSV export	Export route and Python CSV module

11. Testing and Verification

The project contains three test modules: tests/test_app.py, tests/test_frontend.py, and tests/test_inventory_backend.py. These cover application behavior, frontend behavior, and inventory backend logic. Final test counts and coverage percentages should be inserted from the latest successful test and coverage runs.

12. Conclusion

The Electronics Store Inventory Management System uses a Flask-based three-layer web architecture with HTML and CSS presentation, Python application logic, and SQLite persistence. The architecture supports authentication, inventory operations, stock classification, reporting, and CSV export. This document describes the current inspected project structure and identifies deployment and scalability considerations.

13. Review Checklist

- Review the architecture description against the final application code.
- Insert a formal UML component diagram if required by the course template.
- Create and insert UML use-case and sequence diagrams if required as graphical diagrams.
- Verify all route behavior and error handling using the latest test results.
- Add the final performance test results if a measurable performance target is required.
- Review security and deployment settings before any production deployment.
- Update the revision history after instructor or team review.